`04-Express-API/04-put-delete-requests.md`

🔄 PUT & DELETE Requests

🎯 Updating and Deleting Resources

PUT & DELETE - Modify and Remove

Complete CRUD operations

📝 PUT vs PATCH

Understanding the Difference

Method	Purpose	Usage
PUT	Replace entire resource	Update all fields
PATCH	Partial update	Update specific fields

In Practice

```
// PUT - Replace entire course
PUT /api/courses/1
{ "id": 1, "name": "Updated Course", "duration": 10 }

// PATCH - Update just the name
PATCH /api/courses/1
{ "name": "Updated Course" }
```

🔄 PUT Request Steps

Implementation Steps

1. **Look up the resource** (by ID)
2. **Return 404** if not found
3. **Validate input** with Joi
4. **Return 400** if validation fails
5. **Update the resource**
6. **Return updated resource**

Reusing Code

Steps 1-4 are similar to GET and POST!
→ We can create a `validateCourse()` function

✅ Refactoring: Validation Function

Extract Validation Logic

```
function validateCourse(course) {
    const schema = Joi.object({
        name: Joi.string().min(3).required()
    });

    return schema.validate(course);
}
```

Now reuse in both POST and PUT!

Benefits

✅ DRY (Don't Repeat Yourself)
✅ Easier to maintain

✅ Consistent validation
✅ Single source of truth

🔄 Complete PUT Implementation

Full PUT Route

```
app.put('/api/courses/:id', (req, res) => {
    // 1. Look up course
    const course = courses.find(c => c.id === parseInt(req.params.id));

    // 2. Return 404 if not found
    if (!course) {
        return res.status(404).send('Course with given ID not found');
    }

    // 3. Validate input
    const result = validateCourse(req.body);

    // 4. Return 400 if validation fails
    if (result.error) {
        return res.status(400).send(result.error.details[0].message);
    }

    // 5. Update course
    course.name = req.body.name;

    // 6. Return updated course
    res.send(course);
});
```

🧪 Testing PUT with Postman

Update a Course

Method: PUT
URL: `http://localhost:3000/api/courses/1`
Body: raw → JSON

```
{
    "name": "Updated Course Name"
}
```

Click Send

Response (200 OK):

```
{
    "id": 1,
    "name": "Updated Course Name"
}
```

Test Error Cases

Non-existent ID (404):

```
PUT /api/courses/10
→ "Course with given ID not found"
```

Invalid input (400):

```
{ "name": "ab" }
→ "name" length must be at least 3 characters long
```

🔌 Testing PUT with REST Client

Using VS Code REST Client Extension

Create or add to your `test.http` file:

```
### Update a course (PUT)
PUT http://localhost:3000/api/courses/1
Content-Type: application/json

{
    "name": "Updated Course Name"
}

### Update with short name (should fail validation)
PUT http://localhost:3000/api/courses/1
Content-Type: application/json

{
    "name": "ab"
}

### Update non-existent course (should return 404)
PUT http://localhost:3000/api/courses/999
Content-Type: application/json

{
    "name": "This Course Does Not Exist"
}
```

How to Use

1. Install **REST Client** extension in VS Code
2. Click **"Send Request"** above each `PUT` line
3. View response in split pane

Expected Responses

✅ **Valid update:** `200 OK` with updated course
❌ **Short name:** `400 Bad Request` - validation error
❌ **Wrong ID:** `404 Not Found` - course not found

🗑️ DELETE Request

DELETE Steps

1. **Look up the resource** (by ID)
2. **Return 404** if not found
3. **Delete the resource**
4. **Return deleted resource** (convention)

Implementation

```
app.delete('/api/courses/:id', (req, res) => {
    // 1. Look up course
    const course = courses.find(c => c.id === parseInt(req.params.id));

    // 2. Return 404 if not found
    if (!course) {
        return res.status(404).send('Course with given ID not found');
    }

    // 3. Delete course
    const index = courses.indexOf(course);
    courses.splice(index, 1);

    // 4. Return deleted course
    res.send(course);
});
```

⁇ Understanding Array Methods

Array.indexOf()

Returns the position of an item:

```
const courses = [
    { id: 1, name: 'course1' },
    { id: 2, name: 'course2' },
    { id: 3, name: 'course3' }
];

const course = courses.find(c => c.id === 2);
const index = courses.indexOf(course);  // Returns 1
```

Array.splice()

Removes items from array:

```
courses.splice(index, 1);
// splice(startIndex, deleteCount)
```

Before: `[{id:1}, {id:2}, {id:3}]`
After: `[{id:1}, {id:3}]`

🧪 Testing DELETE with Postman

Delete a Course

Method: DELETE
URL: `http://localhost:3000/api/courses/3`

Click Send

Response (200 OK):

```
{
    "id": 3,
    "name": "course3"
}
```

Test Not Found

URL: `http://localhost:3000/api/courses/10`

Response (404 Not Found):

`Course with given ID not found`

🔌 Testing DELETE with REST Client

Using VS Code REST Client Extension

Add to your `test.http` file:

```
### Delete a course
DELETE http://localhost:3000/api/courses/3

### Try to delete non-existent course (should return 404)
DELETE http://localhost:3000/api/courses/999

### Get all courses to verify deletion
GET http://localhost:3000/api/courses
```

How to Use

1. Click **"Send Request"** above the DELETE line
2. View response showing the deleted course
3. Run the GET request to confirm course is removed

Expected Responses

✅ **Valid delete:** `200 OK` with deleted course data
❌ **Wrong ID:** `404 Not Found` - course not found

Tip

💡 After deleting, try to GET the same course ID - it should return 404!

📊 Complete API Summary

All CRUD Operations

Method	Endpoint	Action
GET	/api/courses	Get all courses
GET	/api/courses/:id	Get one course

POST	/api/courses	Create course
PUT	/api/courses/:id	Update course
DELETE	/api/courses/:id	Delete course

🔧 Full API Code

Complete Implementation

```
const express = require('express');
const Joi = require('joi');
const app = express();

app.use(express.json());

const courses = [
    { id: 1, name: 'course1' },
    { id: 2, name: 'course2' },
    { id: 3, name: 'course3' }
];

// GET all
app.get('/api/courses', (req, res) => {
    res.send(courses);
});

// GET one
app.get('/api/courses/:id', (req, res) => {
    const course = courses.find(c => c.id === parseInt(req.params.id));
    if (!course) return res.status(404).send('Not found');
    res.send(course);
});

// POST
app.post('/api/courses', (req, res) => {
    const result = validateCourse(req.body);
    if (result.error) return res.status(400).send(result.error.details[0].message);

    const course = { id: courses.length + 1, name: req.body.name };
    courses.push(course);
```

```
    res.send(course);
});

// PUT
app.put('/api/courses/:id', (req, res) => {
    const course = courses.find(c => c.id === parseInt(req.params.id));
    if (!course) return res.status(404).send('Not found');

    const result = validateCourse(req.body);
    if (result.error) return res.status(400).send(result.error.details[0].message);

    course.name = req.body.name;
    res.send(course);
});

// DELETE
app.delete('/api/courses/:id', (req, res) => {
    const course = courses.find(c => c.id === parseInt(req.params.id));
    if (!course) return res.status(404).send('Not found');

    const index = courses.indexOf(course);
    courses.splice(index, 1);
    res.send(course);
});

function validateCourse(course) {
    const schema = Joi.object({ name: Joi.string().min(3).required() });
    return schema.validate(course);
}

app.listen(3000, () => console.log('Listening on port 3000...'));
```

💡 Best Practices

PUT & DELETE Guidelines

✅ **DO:** Return 404 for non-existent resources
✅ **DO:** Validate input on PUT
✅ **DO:** Return the affected resource
✅ **DO:** Reuse validation functions
✅ **DO:** Use early returns for errors

❌ **DON'T:** Allow deleting without finding first
❌ **DON'T:** Skip validation on updates
❌ **DON'T:** Forget to remove from array (DELETE)
❌ **DON'T:** Use wrong HTTP status codes

🏠 Course Home | 📘 Chapter 4 Home

← Previous: POST Requests | Next: Tools & Best Practices →